

Project 6 - Decision Trees and Random Forest

(200 pts)

1. Project Description:

In this project, you are required to implement **decision trees** and **random forest** as classifiers for the **MNIST dataset** (provided on Harvey) and **one extra dataset** of your own choice (from Kaggle or UCI). You will submit your finished code, a link to the extra dataset, and a thorough project report to receive credits.

Python modules **allowed** for this project are: **numpy, scipy, matplotlib.pyplot, time and concurrent.futures**. Using pre-built decision tree or random forest functions/classes/tools from other Python modules will result in **0 pts** for this project.

2. Project steps

(1) Implement a **single decision tree** for classification tasks based on the C4.5 algorithm.

In our lectures, we introduced the ID3, C4.5 and CART algorithms to build decision trees. You are required to implement the C4.5 algorithm. Pages 17, 23, and 27-30 in the slides for topic 12 part 1 will be helpful to designing a C4.5 tree. Note that this single tree cannot have randomness. Each split, you must consider all the remaining available attributes. The training must be done on the full training set, which means the 60000 images in the MNIST dataset, and 80% of the examples in the extra dataset of your own choice. Your tree's accuracy must be evaluated on the full test set. You can use the practical tricks on page 27 to prune your trees to deal with overfitting.

(2) Implement a random forest with adjustable number of C4.5 trees.

The random forest method is one of the four methods to build ensemble models (topic 12 part 2, page 6-12). Randomness is introduced into the tree building process. First, bootstrap sampling is used to create different sets of examples from the original training set, and each set is used to build one tree. Second, during the splitting of each node, a random subset of the available attributes is selected. InfoGain, splitInfo and gainRatio are computed to help select the best attribute from the subset to split the node. These two levels of randomness must be implemented in your program to make the C4.5 trees diversified and less correlated. Note that C4.5 trees in a random forest do **not** need any pruning! However, you must tune other hyperparameters for better performance: the number of trees in the random forest, the

number of examples in bootstrap sets and the number of randomly selected attributes for splitting.

(3) Collect results and finish the report

3. Useful Tricks

(1) The C4.5 algorithm requires the computation of splitInfo and entropy, both of which might have to deal with $0 * \log_2 0$. Python will automatically assign a nan (not a number) as the result, which cannot be used by subsequent computations. You may use `np.isnan()` to detect nans, and `np.nan_to_num(yourArrayName, nan=0.0)` to replace nans with 0.0.

(2) A special case for C4.5 gainRatio: if splitInfo = 0.0, then gainRatio = 0.0. This avoids division-by-zero.

(3) The MNIST dataset has a relatively high dimension. Each example has 784 pixels (attributes). Each pixel is an integer value in $[0, 255]$. To reduce the execution time of building a full tree, please consider further discretizing the pixel value, which would otherwise create 256 branches at each splitting point. One typical choice is setting a threshold. If a pixel is less than the threshold, replace the pixel with 0; otherwise, replace it with 1. The resulting image will become a binary image. Although such discretization might sacrifice details, it is helpful to reduce computation time and avoid overfitting. You can also consider other ways to handle such large domains.

For your convenience, the 4 parts of the MNIST dataset have been posted on Harvey:

File	Description
train-images-idx3-ubyte	60000 training images
train-labels-idx1-ubyte	60000 training image labels
t10k-images-idx3-ubyte	10000 test images
t10k-labels-idx1-ubyte	10000 test image labels

, and the starter code `RF_starter.py` includes the loading of these parts and displaying of one of the images.

(4) Execution time

For your reference, building a C4.5 tree using the full MNIST training set (of 60000 images) without any pruning and randomness may take about 5 minutes if all the images are converted into binary images first. When you build a full tree in project step 1, to verify your program's correctness, you can first make a subset of the MNIST training set to quickly build a tree. Once confirming that everything is working, you can use the full training set.

For the random forest, a minimum number of 100 trees might be needed to achieve a good accuracy ($\geq 90\%$). You can consider using Python's `concurrent.futures` module to build multiple C4.5 trees in parallel. However, I suggest that you properly set up the number of worker process in this module so that your program will not overload your system.

The following is a sample output for the MNIST dataset from a decently trained random forest with 101 C4.5 trees (index starts with 0, no parallelism). I suggest that you print a message after each tree is built so that you can estimate how long your program will run for.

```
>>>tree 97 is finished
>>>tree 98 is finished
>>>tree 99 is finished
>>>tree 100 is finished
Total training time (seconds): 1066.3944845199585
Accuracy of Random Forest is 0.9055
----- confusion matrix -----
[[ 959    0    1    3    0    1   15    1    0    0]
 [    0 1127    2    2    0    0    4    0    0    0]
 [   13   35  920   10    8    0    8   21   16    1]
 [    1   24   11  927    0    9    3   20   13    2]
 [    2   29    4    0  878    0   12    8    2   47]
 [   14   17    2   76   12  733   21    8    5    4]
 [   14    8    1    0    5    5  925    0    0    0]
 [    2   52   13    2    3    0    1  929    1   25]
 [   14   49    9   70   10   11    7   17  779    8]
 [    9   28    6   12   28    1    4   40    3  878]]
```

(5) Flexibility

You may want to write just one C4.5 tree builder function that can be directed to building either a full tree (without randomness) or a weak tree (as a member of the random forest).

4. Evaluation requirements

(1) For the single full C4.5 tree, tune the parameters that controls the complexity of the tree and report their impacts on accuracy and execution time for the MNIST and your own dataset. (see the practical tricks on page 27 of topic12 part1)

Make a confusion matrix for each dataset. For the MNIST dataset, explain the mistake patterns: how each class is mis-predicted as other classes and what might be the underlying reason (like you did in the Naïve Bayes project)

(2) For the random forest, tune the hyperparameters as mentioned in project step 2. Report their impacts on accuracy and execution time for both datasets.

Make a confusion matrix for each dataset. Compare these new matrices with the single tree's versions and explain what improvements have been made by using the random forest.

5. Submission requirements

(1) Your Python program

(2) The link to your extra dataset

(3) A thorough project report that covers

- How you implemented your C4.5 and random forest. What data structures are used for the nodes and the tree, and why you choose them. How to test a new example using your C4.5 tree.
- All the evaluation requirements in 4.
- Any other interesting discovery in this project.
- A feedback summary of all the project assignments in this course.

The report carries heavy weight of your project score, please make a good one.